libbioestat: biblioteca modular de bioestatística em C

Relatório Técnico do Trabalho Prático 2 (Makefile, bibliotecas estática e dinâmica, testes e Git)

Equipe 22: Gustavo Yamani, Matheus Moreira Cabral, Matheus Guimarães Olegario e Thiago Assumpção Baisch

Outubro de 2026

Sumário

1 Descrição do problema e do domínio escolhido
2 Arquitetura do software
  2.1 Estrutura de diretórios
3 Organização e responsabilidade dos módulos
4 Descrição da API pública
  4.1 Convenção de erros
  4.2 Funções públicas
5 Descrição da aplicação cliente
6 Processo de compilação com Makefile
  6.1 Instalação
7 Estratégia de compilação incremental
8 Geração e utilização da biblioteca estática
9 Geração e utilização da biblioteca dinâmica
10 Comparação entre ligação estática e dinâmica
11 Estratégia e resultados dos testes
  11.1 Estratégia
  11.2 Resultados
12 Exemplo de teste que inicialmente falhou e a correção
13 Estratégia de versionamento com Git
14 Descrição da branch, do merge e da tag v1.0.0
15 Dificuldades encontradas e decisões de projeto
  15.1 Decisões tomadas pelo grupo
  15.2 Dificuldades
16 Resultados obtidos e possíveis melhorias
  16.1 Resultados
  16.2 Possíveis melhorias
  16.3 Thiago Baisch
  16.4 Matheus Olegario
  16.5 Gustavo Yamani
  16.6 Matheus Cabral
17 Documentação dos módulos
  17.1 Módulo de epidemiologia
  17.2 Módulo de testes diagnósticos
  17.3 Módulo de estatística básica
  17.4 Módulo de relações
  17.5 Módulo de visualização

1 Descrição do problema e do domínio escolhido

O Trabalho Prático 2 pedia uma biblioteca reutilizável em C, dividida em módulos, com uma aplicação cliente independente, compilação automática com Make, bibliotecas estática e dinâmica, testes automatizados e um histórico de desenvolvimento em Git.

O domínio que o grupo escolheu foi a **bioestatística**. A biblioteca se chama **libbioestat** e reúne as contas que aparecem com frequência em estudos da área da saúde: medidas de estatística descritiva, medidas de associação entre exposição e doença, qualidade de testes diagnósticos, relação entre duas variáveis e gráficos. A escolha foi natural porque o domínio se divide bem em partes com responsabilidades próprias (e não em arquivos separados só para cumprir a especificação), e porque cada parte tem entradas e saídas bem definidas, o que facilita testar.

O problema que a biblioteca resolve, em uma frase: dado um conjunto de dados (um vetor de números, dois vetores pareados ou uma tabela 2x2 de contagens), calcular as medidas estatísticas e epidemiológicas mais usadas e, quando faz sentido, desenhar o resultado, tudo isso devolvendo erros claros em vez de resultados errados ou travamentos.

Os dados de entrada são de dois tipos:

- **Vetor de números reais** (`double *` e tamanho `size_t`): usado na estatística básica, nas relações entre variáveis e na visualização.
- **Tabela 2x2** (`be_tabela2x2`): usada na epidemiologia e nos testes diagnósticos. Ela cruza a exposição (ou o resultado do teste) com a condição real da pessoa:

	Doente	**Não doente**
Exposto/Teste+	a	b
Não exposto/Teste-	c	d

2 Arquitetura do software

A libbioestat tem 5 módulos de funcionalidade e um módulo de apoio, o de erros. A aplicação cliente fica separada, em `app/`, e usa a biblioteca apenas pelos arquivos `.h` de `include/`.

As dependências entre os módulos são poucas e vão em um único sentido:

- `diagnostico` usa a struct `be_tabela2x2`, que é definida em `epidemiologia.h`.
- `relacoes` reaproveita `be_media` do módulo `basico`.
- `visualizacao` é independente: recebe os dados (e, no boxplot, os quartis) pelos parâmetros e devolve o desenho.
- Todos os módulos usam `bioestat_erro.h`.

2.1 Estrutura de diretórios

```
.
├── include/       Cabeçalhos públicos (a API da biblioteca)
├── src/           Implementação dos módulos
├── app/           Aplicação cliente (menu interativo e --demo)
├── tests/         Testes automatizados (um test_*.c por módulo + teste.h)
├── docs/          Documentação de cada módulo (.md) e este relatório
├── build/         Objetos (.o) e dependências (.d), gerados
├── lib/           libbioestat.a e libbioestat.so, gerados
├── bin/           app_estatica, app_dinamica e os testes, gerados
├── Makefile
├── Doxyfile       Configuração da documentação Doxygen (make doc)
├── README.md
└── demonstacao.mp4
```

3 Organização e responsabilidade dos módulos

Módulo	Responsabilidade	Funções públicas	Responsável
`bioestat_erro`	Convenção de erros comum a toda a biblioteca (`be_status`)	1	Thiago (versão inicial) e Gustavo (`BE_ERRO_MEMORIA`)
`basico`	Estatística descritiva de um vetor: média, mediana, quartis, variância, desvio padrão e moda	8	Gustavo
`relacoes`	Relação entre duas variáveis: covariância, correlação de Pearson, regressão linear e previsão	4	Gustavo
`diagnostico`	Qualidade de um teste diagnóstico: sensibilidade, especificidade, VPP e VPN	4	Matheus Cabral
`epidemiologia`	Tabela 2x2 e medidas de associação: riscos, risco relativo, odds ratio e diferença de riscos	6	Thiago

Módulo	Responsabilidade	Funções públicas	Responsável
`visualizacao`	Gráficos de dispersão, histograma e boxplot, no terminal (ASCII) e em SVG/HTML	6	Matheus Olegario

A documentação completa de cada um (visão geral, decisões, erros, dificuldades e testes) está no capítulo 18.

4 Descrição da API pública

A API pública é o conjunto de funções e tipos declarados em `include/`. São **29 funções** (28 de funcionalidade e `be_status_mensagem`) e 5 tipos (`be_status`, `be_tabela2x2`, `rel_reta`, `Canvas` e `Quartis`, sendo os dois últimos da visualização).

4.1 Convenção de erros

Toda função que pode falhar segue a mesma regra, definida em `bioestat_erro.h`:

1. o **retorno** é sempre um `be_status` (`BE_OK` em caso de sucesso);
2. o **resultado** do cálculo é gravado por um **ponteiro de saída**;
3. nas funções de cálculo, em caso de erro, a saída **não é modificada**.

Código	Quando ocorre
`BE_OK`	Sucesso
`BE_ERRO_PONTEIRO_NULO`	Algum ponteiro obrigatório veio `NULL`
`BE_ERRO_VALOR_INVALIDO`	Entrada fora do domínio (contagem negativa, `n == 0`, etc.)
`BE_ERRO_DIVISAO_POR_ZERO`	O cálculo exigiria dividir por zero
`BE_ERRO_MEMORIA`	Falha de `malloc` / `calloc`
`BE_ERRO_NAO_IMPLEMENTADO`	Reservado para uso durante o desenvolvimento

`be_status_mensagem(status)` converte o código em um texto legível, e a aplicação cliente usa isso para mostrar os erros ao usuário. Um exemplo de uso:

```c
#include "epidemiologia.h"
#include <stdio.h>

int main(void){
    be_tabela2x2 t;
    double rr;

    be_preencher_tabela(20, 80, 10, 90, &t);
    be_status st = be_risco_relativo(&t, &rr);
    if (st != BE_OK) {
        fprintf(stderr, "%s\n", be_status_mensagem(st));
        return 1;
    }
    printf("RR = %.2f\n", rr);   /* RR = 2.00 */
    return 0;
}
```

4.2 Funções públicas

Módulo	Assinatura	O que faz
basico	`be_status be_media(const double *dados, size_t n, double *media)`	Média
basico	`be_status be_mediana(const double *dados, size_t n, double *mediana)`	Mediana
basico	`be_status be_quartil_1(const double *dados, size_t n, double *q1)`	1º quartil
basico	`be_status be_quartil_2(const double *dados, size_t n, double *q2)`	2º quartil
basico	`be_status be_quartil_3(const double *dados, size_t n, double *q3)`	3º quartil
basico	`be_status be_variancia(const double *dados, size_t n, double *var)`	Variância (populacional)
basico	`be_status be_desvio_padrao(const double *dados, size_t n, double *dp)`	Desvio padrão
basico	`be_status be_moda(const double *dados, size_t n, size_t *qtd, double **modas)`	Moda(s); o vetor devolvido é alocado e deve ser liberado por quem chamou

Módulo	Assinatura	O que faz
relacoes	`be_status be_covariancia(const double *dados_x, const double *dados_y, size_t n, double *cov)`	Covariância populacional
relacoes	`be_status be_correlacao(const double *dados_x, const double *dados_y, size_t n, double *corr)`	Correlação de Pearson
relacoes	`be_status be_regressao_linear(const double *dados_x, const double *dados_y, size_t n, rel_reta *reta)`	Reta de regressão (inclinação, intercepto e r^2)
relacoes	`be_status be_prever(const rel_reta *reta, double x, double *y_previsto)`	Valor de y previsto pela reta
diagnostico	`be_status be_sensibilidade(const be_tabela2x2 *t, double *sens)`	Sensibilidade
diagnostico	`be_status be_especificidade(const be_tabela2x2 *t, double *esp)`	Especificidade
diagnostico	`be_status be_vpp(const be_tabela2x2 *t, double *vpp)`	Valor preditivo positivo
diagnostico	`be_status be_vpn(const be_tabela2x2 *t, double *vpn)`	Valor preditivo negativo
epidemiologia	`be_status be_preencher_tabela(long a, long b, long c, long d, be_tabela2x2 *saida)`	Preenche e valida a tabela 2x2
epidemiologia	`be_status be_risco_expostos(const be_tabela2x2 *t, double *saida)`	Risco nos expostos, a/(a+b)
epidemiologia	`be_status be_risco_naoexpostos(const be_tabela2x2 *t, double *saida)`	Risco nos não expostos, c/(c+d)
epidemiologia	`be_status be_risco_relativo(const be_tabela2x2 *t, double *saida)`	Risco relativo (RR)
epidemiologia	`be_status be_odds_ratio(const be_tabela2x2 *t, double *saida)`	Odds ratio (OR)
epidemiologia	`be_status be_diferenca_riscos(const be_tabela2x2 *t, double *saida)`	Diferença de riscos (DR)

Módulo	Assinatura	O que faz
visualizacao	`be_status be_dispersao_ascii(const double *X, const double *Y, int tamanho, Canvas c)`	Dispersão no terminal
visualizacao	`be_status be_histograma_ascii(const double *dados, int tamanho, int num_bins, Canvas c)`	Histograma no terminal
visualizacao	`be_status be_boxplot_ascii(const double *dados, int tamanho, Quartis q, Canvas c)`	Boxplot no terminal
visualizacao	`be_status be_dispersao_svg(const double *X, const double *Y, int tamanho)`	Dispersão em SVG/HTML
visualizacao	`be_status be_histograma_svg(const double *dados, int tamanho, int num_bins)`	Histograma em SVG/HTML
visualizacao	`be_status be_boxplot_svg(const double *dados, int tamanho, Quartis q)`	Boxplot em SVG/HTML
bioestat_erro	`const char *be_status_mensagem(be_status status)`	Texto de um código de erro

Cada função está documentada nos `.h` no formato Doxygen, com finalidade, parâmetros, retorno e condições de erro. Detalhes internos (funções auxiliares como `quantil`, `calcular_somas` ou `normaliza`) são `static` e ficam só nos `.c`. Os cabeçalhos têm *include guards* e usam `const` nos parâmetros que só são lidos.

5 Descrição da aplicação cliente

A aplicação cliente fica em `app/` e é um **menu interativo no terminal**. Ela só usa os `.h` de `include/` e nunca inclui um `.c` da biblioteca. Como a mesma aplicação é ligada à biblioteca estática e à dinâmica, o código-fonte é exatamente o mesmo nos dois casos e só muda o comando de ligação.

Arquivo	Função
`main.c`	Ponto de entrada, leitura básica do teclado, menu principal e menu de entrada de dados. Aceita `--demo`
`dados.c/.h`	Struct `Dados`, que guarda a tabela 2x2 e os vetores carregados, e as funções de leitura e liberação de memória

Arquivo	Função
`menus.h`	Contrato dos menus: protótipos e o padrão visual que todos seguem
`menu_epidemiologia.c`, `menu_diagnostico.c`, `menu_basico.c`, `menu_relacoes.c`, `menu_visualizacao.c`	Um menu e uma demonstração por módulo

Como funciona:

- **Dados compartilhados.** O usuário informa uma tabela 2x2 e/ou vetores uma vez (menu "Entrada de dados") e todos os menus usam esses mesmos dados. Também há uma opção de carregar dados de exemplo.
- **Padrão `ensure_*`.** Se o usuário entra em um módulo sem ter carregado o dado necessário, o menu pergunta se ele quer informar agora (`ensure_table`, `ensure_vector`, etc.).
- **Sem validação duplicada.** A aplicação não valida os dados por conta própria. Se o valor for inválido, ela mostra a mensagem de erro que a própria biblioteca devolveu (`be_status_mensagem`), o que mostra que o tratamento de erros está na biblioteca.
- **Entrada robusta.** A leitura usa `fgets` e `sscanf`, e `Ctrl+D` volta ao menu anterior em vez de travar.
- **Modo `--demo`.** Roda uma demonstração de cada módulo com dados fixos, sem pedir nada. É o que o `make run` executa, porque o menu interativo ficaria esperando o teclado.

```
==========================================
  libbioestat - Biblioteca de Bioestatistica
==========================================
  1) Entrada de dados
  2) Epidemiologia
  3) Testes diagnosticos
  4) Estatistica basica
  5) Relacoes entre variaveis
  6) Visualizacao
  0) Sair
------------------------------------------
```

Todas as 29 funções públicas são usadas pela aplicação cliente ou pelos testes. Os gráficos em SVG são gravados como arquivos `.html` (`grafico_dispersao.html`, `histograma.html` e `boxplot.html`) na pasta em que o programa for executado e podem ser abertos no navegador.

6 Processo de compilação com Makefile

O Makefile foi escrito de forma genérica: ele descobre sozinho os arquivos com `wildcard`, então adicionar um módulo novo em `src/` ou um teste novo em `tests/` não exige mexer

nele.

Alvo	O que faz
`make` / `make all`	Compila a biblioteca (estática e dinâmica) e gera `bin/app_estatica` e `bin/app_dinamica`
`make run`	Compila o que for necessário e executa as duas versões da aplicação com `--demo`
`make test`	Compila e executa todos os testes. Termina com `RESULTADO: TODOS OS TESTES PASSARAM` ou `RESULTADO: HA TESTES FALHANDO` (e código de saída diferente de 0)
`make install [PREFIX=...]`	Instala as duas bibliotecas e os `.h`
`make uninstall [PREFIX=...]`	Remove o que o `install` instalou
`make clean`	Apaga `build/`, `bin/`, `lib/` e `doc/`
`make doc`	Gera a documentação Doxygen em `doc/` (extra)

Flags de compilação: `gcc -Wall -Wextra -std=c11 -Iinclude`. A biblioteca matemática (`-lm`) é ligada porque o projeto usa `sqrt` e outras funções.

Principais regras:

```
# qualquer build/x.o vem de src/x.c (-fPIC porque também vai para a .so)
build/%.o: src/%.c
    mkdir -p build
    $(CC) $(CFLAGS) $(DEPFLAGS) -fPIC -c $< -o $@

# a aplicação não leva -fPIC: ela não vai para a biblioteca
build/app/%.o: app/%.c
    mkdir -p build/app
    $(CC) $(CFLAGS) $(DEPFLAGS) -c $< -o $@
```

Os alvos de teste também usam regras de padrão: todo `tests/test_X.c` vira `bin/tests/test_X`, ligado à biblioteca estática.

6.1 Instalação

```
sudo make install                    # prefixo padrão: /usr/local
make install PREFIX=$HOME/.local     # sem privilégios de administrador
```

O `install` copia `libbioestat.a` e `libbioestat.so` para `$(PREFIX)/lib` e os `.h` para `$(PREFIX)/include/bioestat`. No nosso teste com `PREFIX` em uma pasta qualquer, o resultado foi:

```
inst/include/bioestat/{basico,bioestat_erro,diagnostico,epidemiologia,relacoes,visualizacao}.h
inst/lib/libbioestat.a
inst/lib/libbioestat.so
```

Depois de instalada, a biblioteca pode ser usada por qualquer programa:

```
gcc programa.c -I$PREFIX/include/bioestat -L$PREFIX/lib -lbioestat -lm -o programa
# na execução, com a versão dinâmica, é preciso indicar onde está a .so:
LD_LIBRARY_PATH=$PREFIX/lib ./programa
```

7 Estratégia de compilação incremental

A compilação incremental depende de o `make` saber o que depende de quê. As dependências de `.o` em `.c` ele já conhece pela regra de padrão, mas o problema são os **cabeçalhos**: se `epidemiologia.h` mudar, todo `.c` que o inclui precisa ser recompilado, e o Makefile não sabe disso sozinho.

A solução foi pedir ao próprio `gcc` a lista de cabeçalhos de cada arquivo:

- `-MMD` faz o `gcc` escrever, ao lado de cada `.o`, um arquivo `.d` com os cabeçalhos que o `.c` incluiu;
- `-MP` adiciona alvos vazios para esses cabeçalhos, o que evita erro caso um `.h` seja apagado ou renomeado;
- no final do Makefile, `-include $(LIB_OBJS:.o=.d) $(APP_OBJS:.o=.d) $(TEST_OBJS:.o=.d)` carrega esses `.d` (o `-` ignora a ausência deles na primeira compilação).

Um alvo que depende de uma lista (`$(LIB_OBJS)`) só é refeito se algum item da lista ficou mais novo, então a biblioteca e os executáveis também são refeitos só quando preciso.

Testamos dois cenários depois de uma compilação completa:

1. Alteração em `src/basico.c`. Só esse arquivo foi recompilado, e depois disso foram refeitos apenas os alvos que dependem dele (`libbioestat.a`, `libbioestat.so` e os dois executáveis):

```
$ touch src/basico.c && make
gcc ... -c src/basico.c -o build/basico.o
ar rcs lib/libbioestat.a ...
gcc ... -o bin/app_estatica
gcc -shared -o lib/libbioestat.so ...
gcc ... -o bin/app_dinamica
```

2. Alteração em `include/epidemiologia.h`. Foram recompilados apenas os arquivos que incluem esse cabeçalho: `src/epidemiologia.c`, `src/diagnostico.c` (que usa a `be_tabela2x2`) e os

sete `.c` de `app/` (por causa do `dados.h`). Os módulos `basico`, `relacoes` e `visualizacao` não foram recompilados.

8 Geração e utilização da biblioteca estática

A biblioteca estática, `lib/libbioestat.a`, é um arquivo (um "pacote") que reúne os `.o` de todos os módulos da biblioteca, **sem** os objetos da aplicação (não existe `main` dentro dela).

```
$(STATIC_LIB): $(LIB_OBJS)
    mkdir -p lib
    ar rcs $@ $^

bin/app_estatica: $(APP_OBJS) $(STATIC_LIB)
    mkdir -p bin
    $(CC) $(APP_OBJS) $(STATIC_LIB) $(LDLIBS) -o $@
```

- `ar rcs` cria o arquivo `.a` (`r` insere os objetos, `c` cria, `s` gera o índice de símbolos).
- Na ligação, o `gcc` copia para o executável as funções da biblioteca que a aplicação realmente usa. Por isso o `bin/app_estatica` roda sem precisar de nenhum arquivo `libbioestat` na máquina.
- Os testes (`bin/tests/test_*`) também são ligados à biblioteca estática.

Para confirmar, usamos `ldd bin/app_estatica`: nenhuma biblioteca `libbioestat` aparece entre as dependências.

9 Geração e utilização da biblioteca dinâmica

A biblioteca dinâmica, `lib/libbioestat.so`, é gerada a partir dos mesmos `.o` (compilados com `-fPIC`, ou seja, código que não assume endereços fixos e pode ser carregado em qualquer lugar da memória).

```
$(SHARED_LIB): $(LIB_OBJS)
    mkdir -p lib
    $(CC) $(SHARED_FLAGS) -o $@ $^ $(LDLIBS)      # SHARED_FLAGS = -shared

bin/app_dinamica: $(APP_OBJS) $(SHARED_LIB)
    mkdir -p bin
    $(CC) $(APP_OBJS) -Llib -lbioestat $(RPATH) $(LDLIBS) -o $@
```

- `-Llib` diz **onde** procurar a biblioteca na hora de ligar e `-lbioestat` diz **qual** (procura `libbioestat.so`).
- `-Wl,-rpath,'$ORIGIN/../lib'` grava dentro do executável o caminho onde a `.so` estará **na execução**, relativo ao próprio executável (`bin/../lib`). Com isso o programa roda sem

precisar exportar `LD_LIBRARY_PATH`.
- No macOS (onde o Thiago desenvolveu e testou o Makefile), o Makefile detecta o sistema com `uname -s` e troca a extensão para `.dylib`, as flags para `-dynamiclib -install_name @rpath/...` e o rpath para `@executable_path/../lib`. No Linux, que é o ambiente exigido, ele usa `.so`.

Evidência de que a biblioteca dinâmica é de fato usada na execução. O `ldd` mostra que o executável depende da `.so` e a encontra pelo rpath:

```
$ ldd bin/app_dinamica
    libbioestat.so => .../bin/../lib/libbioestat.so
    libm.so.6      => /lib/.../libm.so.6
```

Como o código da aplicação é o mesmo nas duas versões, as saídas do `--demo` são idênticas, e conferimos isso comparando o `md5sum` das duas saídas.

10 Comparação entre ligação estática e dinâmica

	Estática (`.a`)	**Dinâmica (`.so`)**
Quando a biblioteca é ligada	Em tempo de ligação (`ld`): o código usado é copiado para o executável	Em tempo de execução: o carregador dinâmico (`ld.so`) liga o executável à `.so` ao iniciar
Tamanho do executável	Maior, pois carrega uma cópia do código	Menor, pois só guarda a referência
Dependências na execução	Nenhuma da libbioestat: o executável é autossuficiente	A `.so` precisa ser encontrada (rpath, `LD_LIBRARY_PATH` ou diretório do sistema)
Atualizar a biblioteca	É preciso religar (recompilar) cada programa	Basta trocar a `.so`, sem religar, desde que a interface não mude
Memória	Cada processo tem sua cópia	Várias aplicações compartilham o mesmo código na memória
Compilação do código da lib	Sem exigência especial	Precisa de `-fPIC`
Distribuição	Simples (um arquivo só)	Mais cuidadosa (programa e `.so` juntos)

No nosso projeto, os tamanhos medidos em Linux (aarch64) foram: `app_estatica` com 59.552 bytes e `app_dinamica` com 45.992 bytes, sendo que a `libbioestat.so` tem 35.568 bytes e a `libbioestat.a` tem 40.350 bytes. Ou seja, o executável dinâmico é menor, mas só funciona com a `.so` ao lado. Usamos a estática nos testes por ser autossuficiente.

11 Estratégia e resultados dos testes

11.1 Estratégia

- **Um arquivo de teste por módulo** em `tests/` (`test_basico.c`, `test_diagnostico.c`, `test_epidemiologia.c`, `test_relacoes.c` e `test_visualizacao.c`), então todo módulo tem pelo menos um caso específico e todas as funções públicas são testadas.
- **Mini-framework próprio** (`tests/teste.h`), sem dependências externas, com as macros `VERIFICA(cond, "descrição")`, `VERIFICA_PROXIMO(obtido, esperado, "descrição")` (compara `double` com tolerância de 1e-9) e `RESULTADO_TESTES()`, que devolve 0 ou 1 como código de saída do programa. O `make test` usa esse código para dizer se passou ou falhou. O módulo de visualização usa `assert`.
- **Categorias de teste em todos os módulos:**
  - casos normais, com valores conferidos à mão;
  - casos de borda (`n == 1`, `n == 2`, valores iguais, zeros, vetores grandes, contagens grandes, valores negativos);
  - ponteiros nulos;
  - valores inválidos (negativos, `n == 0`);
  - divisão por zero;
  - saída intacta em caso de erro: a variável de saída começa com um valor sentinela (por exemplo `-9.0`) e o teste confere que ele continua igual depois do erro.
- **Visualização:** os testes conferem se o desenho ASCII sai como esperado e se os arquivos SVG/HTML são criados e não estão vazios.

11.2 Resultados

`make test` na versão final (commit mais recente):

Teste	Verificações
`test_basico`	162/162
`test_diagnostico`	13/13
`test_epidemiologia`	75/75
`test_relacoes`	41/41
`test_visualizacao`	todos os `assert` passam

```
RESULTADO: TODOS OS TESTES PASSARAM
```

12 Exemplo de teste que inicialmente falhou e a correção

O exemplo é o **overflow no `be_odds_ratio`**, do módulo de epidemiologia. O OR é `(a * d) / (b * c)` e as contagens da tabela são `long`. A primeira versão fazia a multiplicação em inteiros e só depois convertia para `double`:

```
        if (t->b * t->c == 0) return BE_ERRO_DIVISAO_POR_ZERO;
        *saida = (double)(t->a * t->d) / (t->b * t->c);
```

Com contagens grandes, `a * d` estoura o `long` (64 bits) antes da conversão. Há ainda um segundo defeito: `b * c` pode dar `0` por estouro mesmo com `b` e `c` diferentes de zero (por exemplo, `b = c = 2^32`), o que faria a função devolver erro de divisão por zero sem motivo.

O teste que pega o problema usa a tabela `{4000000000, 1, 1, 4000000000}`, para a qual o OR esperado é `1,6e19`:

```
        be_tabela2x2 grande = {4000000000L, 1, 1, 4000000000L};
        r = -9.0;
        st = be_odds_ratio(&grande, &r);
        VERIFICA(st == BE_OK, "odds_ratio: contagens grandes devolve BE_OK");
        VERIFICA_PROXIMO(r, 1.6e19, "odds_ratio: contagens grandes sem overflow");
```

Antes da correção (voltando a linha do cálculo para a versão do commit anterior ao `d325660` e rodando `make test`):

```
[FALHA] odds_ratio: contagens grandes sem overflow  (tests/test_epidemiologia.c:178)
-> 74/75 verificacoes passaram
```

Correção (commit `d325660`, “correção de overflow na função odds_ratio (convertendo tudo para double) e simplificação das verificações de erro dessa função”): a divisão por zero passou a ser verificada diretamente em `b` e `c`, e cada fator é convertido para `double` antes da multiplicação.

```
        if (t->b == 0 || t->c == 0) return BE_ERRO_DIVISAO_POR_ZERO;
        *saida = ((double)t->a * (double)t->d) / ((double)t->b * (double)t->c);
```

Depois da correção:

```
-> 75/75 verificacoes passaram
```

Para repetir a demonstração: trocar a linha do cálculo pela versão antiga, rodar `make test` (o teste falha) e restaurar a versão corrigida (o teste passa).

Um segundo caso de falha encontrada nos testes aconteceu no módulo de relações, com séries constantes: testar `den == 0.0` não era suficiente por causa de arredondamentos, e a função `constante`, que faz uma comparação exata com os valores, foi criada para isso (detalhes na documentação do módulo de relações).

13 Estratégia de versionamento com Git

O repositório fica no GitHub (`omatheusolegario/mac0216-biblioteca-bioestat-equipe22`) e teve, do primeiro ao último dia (12 a 19 de outubro de 2026), **113 commits**.

- **Uma branch por pessoa/funcionalidade.** Cada integrante desenvolveu o seu módulo em uma branch própria (`olegario`, `thiago`, `thiago-epidemio`, `matheusmcabral` e `gustavo`) e integrou na `main` com merge quando o módulo estava funcionando. Trabalhos de integração tiveram branches de funcionalidade: `feature-menu-visualizacao`, `features-visualizacao` e `features-gerais`.
- **Commits pequenos e incrementais.** Por exemplo, a epidemiologia nasceu com o esqueleto da API (`f177010`), depois a primeira função e seu teste (`629f518`, `7ce79fb`), depois as demais funções (`456605d`), a correção do overflow (`d325660`) e a limpeza do código. As mensagens dizem o que mudou.
- **Padronização por commits dedicados.** Quando o grupo definiu a convenção de erros (`be_status`), houve commits específicos para adaptar cada módulo (por exemplo `b9fb854`, `ec891be` e `16c67ec`).
- **`.gitignore`** para `build/`, `lib/`, `bin/`, `doc/`, objetos e arquivos de editor, de modo que só o código-fonte vai para o repositório.

14 Descrição da branch, do merge e da tag v1.0.0

Branches. Além da `main`: `thiago`, `thiago-epidemio`, `olegario`, `matheusmcabral`, `gustavo`, `feature-menu-visualizacao`, `features-visualizacao` e `features-gerais`.

Merges. Todos os módulos foram integrados à `main` por merge, e os *merge commits* aparecem no `git log --graph`:

Commit	Merge	Data
`757c435`	Pull request #1 (`thiago`) na `main`	15/10
`d802681`	`olegario` (módulo de visualização)	15/10
`ff02268`	`matheusmcabral` (módulo de diagnóstico)	15/10
`a4f877c`	`thiago-epidemio` (epidemiologia e aplicação cliente)	15/10

Commit	Merge	Data
`f033a88`	`olegario` (documentação do módulo de visualização)	15/10
`57421e2`	`gustavo` (módulos básico e relações)	16/10
`fc0aa3c`	`features-gerais` (README, Doxyfile e `make doc`)	17/10
`d85c0ea`	`features-visualizacao` (correlação nos gráficos de dispersão)	17/10

Para demonstrar a branch e o merge: `git branch -a` lista as branches e `git log --graph --oneline --all` mostra os caminhos separados e os pontos de merge.

```
git tag -n          # lista as tags
git show v1.0.0     # mostra a tag e o commit
```

15 Dificuldades encontradas e decisões de projeto

15.1 Decisões tomadas pelo grupo

1. **Domínio e nome.** Bioestatística, com a biblioteca chamada `libbioestat` e prefixo `be_` em todas as funções públicas, para evitar conflito de nomes com outros códigos.
2. **Divisão em módulos por responsabilidade.** Estatística básica, relações entre variáveis, testes diagnósticos, epidemiologia e visualização. Cada módulo responde a uma pergunta diferente do domínio e pode ser usado separadamente.
3. **Convenção única de erros.** Toda função devolve `be_status` e grava o resultado em um ponteiro de saída. Em erro, a saída não é modificada. A mesma ordem de verificação foi seguida: ponteiros nulos, valores inválidos e divisão por zero.
4. **Tabela 2x2 compartilhada.** A `be_tabela2x2` fica em `epidemiologia.h` e é reaproveitada pelo módulo de diagnóstico, em vez de ter duas structs iguais.
5. **Reaproveitar em vez de repetir.** `relacoes` usa `be_media` do módulo básico, `be_regressao_linear` reaproveita o cálculo da correlação, `be_risco_relativo` e `be_diferenca_riscos` reaproveitam as funções de risco e mediana e segundo quartil usam a mesma função `quantil`.
6. **Escolhas estatísticas.** Variância e covariância populacionais (dividem por n). Quartis por interpolação linear. Moda pode ter vários valores, então devolve um vetor alocado (quem chama libera).
7. **Funções de visualização sem nome de arquivo.** Os parâmetros de nome de arquivo das funções SVG foram retirados para deixar a assinatura mais simples (os arquivos saem com nomes fixos), o que exigiu adaptar os menus.

8. **Aplicação só com a API pública.** `app/` inclui apenas os `.h` de `include/`. Os dados que o usuário digita ficam em uma struct própria da aplicação (`Dados`), que não faz parte da biblioteca.
9. **Mini-framework de testes próprio (`teste.h`).** Evita dependência externa e permite o `make test` com código de saída.
10. **Makefile genérico.** Descobre arquivos por `wildcard` e gera dependências de cabeçalhos com `-MMD -MP`.
11. **Instalação sem administrador.** `PREFIX` configurável, e instruções no README para instalar e desinstalar em `$HOME/.local`.
12. **Extras.** Documentação Doxygen (`make doc`), modo `--demo`, README completo e vídeo de demonstração.

15.2 Dificuldades

- **Definir o tratamento de erros tarde.** No começo cada módulo devolvia o resultado diretamente. Quando o grupo definiu o `be_status` com ponteiro de saída, as funções de todos os módulos tiveram de ser adaptadas (houve commits de padronização no `basico`, no `diagnostico` e na `visualizacao`), e o retorno que antes era o resultado passou a ser o status.
- **Mudar assinaturas no meio do projeto.** Ao mudar a assinatura de funções de visualização, os menus da aplicação quebraram e foi preciso corrigi-los em seguida. Isso mostrou a importância de combinar a API antes de os outros dependerem dela.
- **Compilação incremental com cabeçalhos.** Descobrir como o `make` perceberia a alteração de um `.h` e chegar na solução com `-MMD -MP`.
- **Linux e macOS.** Parte do desenvolvimento foi feita em macOS (Thiago), mas o ambiente exigido é Linux. A biblioteca dinâmica tem extensão e flags diferentes no macOS, então o Makefile precisou detectar o sistema para continuar funcionando no Linux, que é o ambiente exigido.
- **Gerenciamento de memória.** Erros com `calloc`/`malloc` e liberação no momento errado na visualização, variáveis não inicializadas com lixo na memória e a correção do gerenciamento de memória em `dados.c`.
- **Casos de borda numéricos.** Divisão por zero, séries constantes, arredondamento de ponto flutuante (relações) e overflow de inteiros (epidemiologia).
- **Integração de várias branches.** Os módulos foram feitos em paralelo e integrados em momentos diferentes, e foi preciso manter os menus e os nomes consistentes entre eles (commits de “correções ao longo de todo código para garantir consistência dos menus”).
- **Desenhar gráficos em ASCII e SVG.** Pensar em coordenadas (linhas e colunas na matriz, eixos relativos à origem, escala) de forma geral para qualquer valor. Mais detalhes na documentação do módulo de visualização.

16 Resultados obtidos e possíveis melhorias

16.1 Resultados

- Biblioteca com **29 funções públicas** em **5 módulos de funcionalidade** (mais o módulo de erros), cada função documentada e testada.
- Aplicação cliente interativa que usa só a API pública e funciona ligada tanto à biblioteca estática quanto à dinâmica, com saídas idênticas.
- Makefile com `all`, `clean`, `run`, `test`, `install` (com `PREFIX`), `uninstall` e `doc`, e compilação incremental que inclui as dependências de cabeçalhos.
- **291 verificações automáticas** nos módulos básico, diagnóstico, epidemiologia e relações, mais os testes de visualização, todos passando.
- Histórico Git com 113 commits, 8 branches, 8 merges principais e a tag `v1.0.0`.
- Documentação Doxygen nos `.h`, documentação de cada módulo, README, vídeo de demonstração e este relatório.

16.2 Possíveis melhorias

- **Mais medidas de epidemiologia:** intervalos de confiança para o RR e o OR, teste qui-quadrado e o número necessário para tratar (NNT).
- **Variância amostral** (dividir por n - 1) como opção, já que é o que se usa quando os dados são uma amostra. Também seria bom revisar o README, que cita “variância amostral” enquanto a implementação é a populacional.
- **Padronizar os tipos de tamanho:** a visualização usa `int` para o tamanho dos vetores e os outros módulos usam `size_t`
- **Gráficos adicionais** (barras, linhas, curva da regressão no SVG) e leitura de dados a partir de arquivos CSV.

16.3 Thiago Baisch

- **Módulo de epidemiologia** completo: `epidemiologia.h`, `epidemiologia.c` e `test_epidemiologia.c` (tabela 2x2, riscos, risco relativo, odds ratio e diferença de riscos), com a correção do overflow do odds ratio. Documentação do módulo.
- **Estrutura inicial do projeto** (esqueleto de pastas) e **Makefile completo**: biblioteca estática e dinâmica, adaptação para macOS, compilação incremental com `-MMD -MP`, alvos `run`, `test`, `install` e `uninstall`.
- **Mini-framework de testes** (`tests/teste.h`) e o alvo `make test`.
- **Base da aplicação cliente:** contrato dos menus (`menus.h`), contrato da entrada de dados (`dados.h` e `dados.c`), funções de leitura do teclado, `main.c` com menu principal e `--demo`, menu de epidemiologia e o esqueleto dos menus dos outros módulos, que os demais integrantes completaram.
- Arquivo `bioestat_erro` na sua versão inicial.
- Relatório final do projeto

16.4 Matheus Olegario

- **Módulo de visualização** completo: `visualizacao.h`, `visualizacao.c` (o maior módulo, com 778 linhas) e `test_visualizacao.c`. Gráficos de dispersão, histograma e boxplot em ASCII e em SVG/HTML, incluindo eixos relativos à origem, legendas e a correlação linear nos gráficos de dispersão. Documentação do módulo.
- **Menu e demonstração de visualização** e as correções de gerenciamento de memória em `dados.c`.
- **README.md**, **Doxyfile** e o alvo `make doc`, além das instruções de instalação com `PREFIX`.
- **Tag** `v1.0.0` e vários merges de integração das branches, incluindo o ajuste de consistência entre os menus.

16.5 Gustavo Yamani

- **Módulo básico** completo: `basico.h`, `basico.c` e `test_basico.c` (média, mediana, quartis, variância, desvio padrão e moda).
- **Módulo de relações** completo: `relacoes.h`, `relacoes.c` e `test_relacoes.c` (covariância, correlação, regressão linear e previsão).
- **Menus** dos módulos básico e relações na aplicação cliente.
- `BE_ERRO_MEMORIA` no tratamento de erros, padronização do módulo básico e da documentação Doxygen e documentação dos dois módulos.

16.6 Matheus Cabral

- **Módulo de diagnóstico** completo: `diagnostico.h`, `diagnostico.c` e `test_diagnostico.c` (sensibilidade, especificidade, VPP e VPN), com a adaptação ao tratamento de erros. Documentação do módulo.
- **Menu do módulo de diagnóstico** na aplicação cliente.
- Ajustes na documentação Doxygen (por exemplo o `ifndef` nos cabeçalhos, a clareza sobre teste positivo ou negativo na tabela 2x2 e a tabela formatada).
- **Vídeo de demonstração** do projeto (`demonstacao.mp4`).

17 Documentação dos módulos

Esta seção reúne a documentação de cada módulo, escrita pelo responsável por ele. Os mesmos textos estão na pasta `docs/` do repositório.

17.1 Módulo de epidemiologia

Thiago Assumpção Baisch - 11229110

17.1.1 Visão geral do Módulo

O objetivo desse módulo é comparar o risco de adoecer entre um grupo exposto e um grupo não exposto a algum fator. Para todas as funções, é dada uma tabela 2x2 que cruza a exposição da pessoa com o fato dela estar doente ou não.

A tabela segue o formato abaixo (o mesmo usado na documentação do código):

	Doente	**Não doente**
Exposto/Teste+	a	b
Não exposto/Teste-	c	d

17.1.2 O que foi desenvolvido

- Structs:
  - be_tabela2x2, para agrupar as quatro contagens (a, b, c, d) da tabela. Os campos são long, porque são contagens de pessoas e podem ser grandes.
- Funções de epidemiologia:
  - be_preencher_tabela: Preenche uma be_tabela2x2 a partir de quatro contagens, validando que nenhuma é negativa.
  - be_risco_expostos: Calcula a probabilidade de adoecer entre os expostos (incidência cumulativa dos expostos), a / (a + b).
  - be_risco_naoexpostos: Calcula a probabilidade de adoecer entre os não expostos, c / (c + d).
  - be_risco_relativo: Calcula o risco relativo (RR), risco dos expostos dividido pelo risco dos não expostos. RR = 1 significa sem associação, RR > 1 a exposição aumenta o risco e RR < 1 a exposição protege.
  - be_odds_ratio: Calcula a razão de chances (OR), (a * d) / (b * c).
  - be_diferenca_riscos: Calcula a diferença de riscos (DR), risco dos expostos menos risco dos não expostos. Diz quantos casos a mais (ou a menos) a exposição causa em termos absolutos.
- Menu da aplicação (app/menu_epidemiologia.c):
  - menu_epidemiologia: Menu interativo para informar a tabela e calcular cada medida, ou todas de uma vez.
  - demo_epidemiologia: Demonstração usada pelo –demo, com a tabela {20, 80, 10, 90}.

17.1.3 Decisões de arquitetura e design

- A be_tabela2x2 é a entrada padrão de todo o módulo. Ela também é reutilizada pelo módulo de diagnóstico, por isso ela fica definida em epidemiologia.h.
- Todas as funções seguem a convenção da biblioteca: o retorno é o be_status e o resultado é gravado em um ponteiro de saída (double *saida).

- A tabela é passada como const be_tabela2x2 *, já que as funções só leem os valores. Só be_preencher_tabela escreve na struct.
- be_risco_relativo e be_diferenca_riscos reaproveitam be_risco_expostos e be_risco_naoexpostos, em vez de repetir as fórmulas. Se uma das chamadas internas falha, o status dela é repassado.
- Nas divisões entre contagens, o numerador é convertido para double antes de dividir. Sem isso, a divisão seria inteira e o resultado ficaria errado (4 / 9 daria 0).
- No odds ratio, a e d (e b e c) são convertidos para double antes da multiplicação. Como são long, o produto a * d poderia estourar para contagens grandes.
- be_diferenca_riscos não testa divisão por zero diretamente, isso já é feito dentro das funções de risco.
- Os resultados só são escritos em *saida quando está tudo certo, então em caso de erro a saída fica como estava.

17.1.4 Tratamento de erros e casos de borda

- Todas as funções verificam, nessa ordem: ponteiros nulos, valores negativos na tabela e divisão por zero.
- Divisão por zero em cada função:
  - be_risco_expostos: a + b == 0 (grupo dos expostos vazio).
  - be_risco_naoexpostos: c + d == 0 (grupo dos não expostos vazio).
  - be_risco_relativo: a + b == 0, c + d == 0 ou c == 0. O último caso é quando o risco dos não expostos é zero, ou seja, o denominador da razão.
  - be_odds_ratio: b == 0 ou c == 0.
  - be_diferenca_riscos: a + b == 0 ou c + d == 0.
- Não há erro de memória, já que esse módulo não aloca memória.
- Casos de borda tratados:
  - Tabela com zeros: be_preencher_tabela aceita 0 em qualquer célula.
  - a == 0: Risco dos expostos é 0. RR e OR valem 0 (se o outro grupo permite o cálculo).
  - b == 0: Risco dos expostos é 1. RR é válido, já que b não está no denominador do RR. Já o OR devolve erro de divisão por zero.
  - c == 0: Risco dos não expostos é 0. RR e OR devolvem erro de divisão por zero.
  - Riscos iguais: RR vale 1 e DR vale 0.
  - Exposição protetora: DR negativa e RR menor que 1 são resultados válidos, não são erros.
  - Grupo vazio (a + b == 0 ou c + d == 0): Divisão por zero nas medidas que dependem do risco daquele grupo.
  - Contagens muito grandes: OR calculado sem overflow.
  - Saídas intactas em caso de erro.

17.1.5 Dificuldades encontradas

- Retorno das funções ocupado pelo be_status. As funções tiveram que ser adaptadas para gravar o resultado em um ponteiro de saída.
- Descobrir exatamente quando o RR e o OR têm divisão por zero. No RR, o denominador é o risco dos não expostos, então só c == 0 (e não b == 0) causa problema. No OR é o contrário, tanto b quanto c zerados dão erro. Foi preciso pensar caso a caso.
- Divisão inteira. Com os campos sendo long, esquecer de converter para double faz o risco virar 0 ou 1.
- Overflow no odds ratio com contagens grandes, que resolvemos convertendo para double antes de multiplicar.
- Decidir o que é erro e o que é só um resultado incomum. Por exemplo, DR negativa é válida (exposição protetora) e não deve devolver erro.

17.1.6 Testes

- Existem testes para cada função desse módulo em tests/test_epidemiologia.c.
- Os testes abordam:
    - Casos com entrada normal, usando uma tabela de referência {20, 80, 10, 90} (risco dos expostos 0,2; risco dos não expostos 0,1; RR = 2; OR = 2,25; DR = 0,1) e uma tabela com DR negativa;
    - Casos de borda (a, b ou c zerados, riscos iguais, risco 0 e risco 1, contagens grandes);
    - Casos com ponteiro nulo (tabela e saída);
    - Casos com valor inválido (valor negativo em cada uma das células);
    - Casos de divisão por zero (grupos vazios, b == 0, c == 0);
- Nos testes de erro, a variável de saída começa com o valor -9.0, para verificar que ela não foi modificada quando a função devolve erro.

17.2 Módulo de testes diagnósticos

17.2.1 Visão geral do Módulo

O objetivo desse módulo é avaliar a qualidade de um teste diagnóstico. Para todas as funções, é dada uma tabela 2x2 que cruza a condição real da pessoa com o resultado do teste.

17.2.2 O que foi desenvolvido

- Funções de diagnóstico
    - be_sensibilidade: Calcula a proporção das pessoas indentificadas como positivas, dentre as que estão verdadeiramente doente. Assim, avalia a capacidade do teste indentificar quem tem a doença.
    - be_especificidade: Calcula a proporção das pessoas indentificadas como negativas, dentre as que verdadeiramente não estão doentes. Assim, avalia a capacidade do

teste indentificar quem não tem a doença.
    - be_vpp: Valor preditivo positivo - calcula a probabilidade da pessoa estar doente, dado que o teste deu positivo.
    - be_vpn: Valor preditivo negatico - calcula a probabilidade da pessoa não estar doente, dado que o teste deu negativo.

17.2.3 Decisões de arquitetura e design

- Todas as funções do módulo utilizam a struct be_tabela2x2 criada no módulo epidemiologia.
- Como o retorno de cada função é o status de se houve erro ou não, cada função também recebe um ponteiro e escreve nesse endereço o valor calculado pela função.

17.2.4 Tratamento de erros e casos de borda

- Todas as funções do módulo verificam, antes de fazer o cálculo, se:
    - Tem algum ponteiro nulo.
    - Tem algum valor negativo na tabela.
    - O cálculo exigiria uma divisão por zero.
- Em qualquer um desses casos, é retornado um código de erro apropriado.

17.2.5 Dificuldades encontradas

- Inicialmente, não tinhamos determinado como o tratamento de erro seria feito. As funções tiveram que ser adaptadas para retornar o be_status e escrever o resultado em um endereço de memória dado por um ponteiro.

17.2.6 Testes

- Existem testes para cada função desse módulo em tests/test_diagnostico.c, além de testes com valores normais, tem testes que verificam se as funções retornam o código de erro apropriado em casos com entrada inválida.

17.3 Módulo de estatística básica

17.3.1 Visão geral do Módulo

O objetivo desse módulo é reunir as medidas básicas de estatística sobre um conjunto de números reais. Como ele contém essas medidas básicas, esse módulo é utilizado como um módulo de apoio, em especial, para o módulo de relações.

17.3.2 O que foi desenvolvido

- Funções de estatística básica:
    - be_media: Calcula a média.

- be_mediana: Calcula a mediana (quantil de ordem 0,5).
- be_quartil_1: Calcula o primeiro quartil (quantil de ordem 0,25).
- be_quartil_2: Calcula o segundo quartil (quantil de ordem 0,5).
- be_quartil_3: Calcula o terceiro quartil (quantil de ordem 0,75).
- be_variancia: Calcula a variância.
- be_desvio_padrao: Calcula o desvio padrão (raiz quadrada da variância).
- be_moda: Calcula a moda.

- Funções auxiliares (privadas):
  - comparar: Compara dois double.
  - dados_ordenados: Devolve uma cópia ordenada do vetor de entrada.
  - quantil: Calcula o quantil de ordem p por interpolação linear. p vale entre 0 e 1.
  - tam_sequencia: Verifica o tamanho da sequência de valores iguais que começa a partir de determinada posição do vetor já ordenado.

17.3.3 Decisões de arquitetura e design

- Mediana, quartis e moda precisam dos dados ordenados, mas o parâmetro é const double *. Por conta disso, dados_ordenados ordena uma cópia.
- Utiliza-se a função auxiliar quantil para os quartils e mediana, com p diferente, para evitar código repetido e garantir que a mediana e o segundo quartil coincidam. Para o cálculo, o método escolhido foi a interpolação linear.
- A variância calculada é a variância populacional (divide-se por n).
- Funções como be_media e be_variancia são reaproveitadas.
- be_moda passa duas vezes pelo vetor ordenado. A primeira passada descobre a maior frequência e quantos valores a possuem. Com esses dados, aloca-se um vetor com o tamanho exato. A segunda passada preenche esse vetor.
- Como a moda pode ser mais que um valor, be_moda devolve um vetor alocado com malloc em `*modas` e a quantidade de valores em `*qtd`.
- be_moda somente aloca. Quem chama be_moda deve liberar.

17.3.4 Tratamento de erros e casos de borda

- Antes dos cálculos serem realizados, os ponteiros e então os valores são verificados.
- Não há erro de divisão por zero, já que n == 0 é rejeitado na verificação de valores.
- Há a verificação de erro de memória sempre que há uso de malloc.
- Casos de borda tratados
  - n == 1: Média, mediana, quartis e moda são o próprio valor. Variância e desvio padrão valem 0.
  - Todos os valores iguais (n == 1 cai dentro desse caso): Variância e desvio padrão valem 0. Moda é esse valor.
  - n == 2: Quartis são interpolados.

- Moda com empate: Moda é todos os valores de maior frequência, ordenados, com cada valor distinto aparecendo uma única vez.
- Moda sem repetições: Moda é todos os valores, ordenados.
- Valores negativo
- Vetor grande
- Dados desordenados
- Saídas intactas em caso de erro

17.3.5 Dificuldades encontradas

- Retorno das funções ocupado pelo be_status. Não era mais possível retornar o resultado diretamente, foi necessário criar um parâmetro de saída e trabalhar com ponteiros.
- Moda com vários valores.

17.3.6 Testes

- Os testes abordam:
  - Casos com entrada normal;
  - Casos de borda;
  - Casos com ponteiro nulo;
  - Casos com valor inválido;
- Em test_basico há a utilização de um struct e funções auxiliares. Como todas as funções do módulo, menos be_moda possuem mesmos parâmetros, elas são verificadas juntas.
- be_moda é testada separadamente.

17.4 Módulo de relações

17.4.1 Visão geral do Módulo

O objetivo desse módulo é medir a relação entre duas variáveis representadas por séries de dados pareadas.

17.4.2 O que foi desenvolvido

- Structs:
  - rel_reta, para agrupar inclinacao, intercepta e r2.
  - somas, para agrupar as médias de cada variável, sxx (soma de (x - média de x)^2), syy (soma de (y - média de y)^2) e sxy (soma de (x - média de x) * (y - média de y)).
- Funções de relações:
  - be_covariancia: Calcula a covariância populacional (sxy / n).
  - be_correlacao: Calcula a correlação de Pearson (sxy / sqrt(sxx * syy))
  - be_regressao_linear: Faz uma regressão linear simples.

- be_prever: Calcula o valor de y previsto por uma reta de regressão.

- Funções auxiliares (privadas):
  - calcular_somas: Valida as séries e preenche uma somas.
  - constante: Informa se todos os valores de um vetor são iguais.
  - calcular_corr: Calcula a correlação, verifica se há divisão por zero e conserta erros de arredondamento.

17.4.3 Decisões de arquitetura e design

- As médias e as somas dos desvios são calculadas uma única vez em calcular_somas. Isso simplifica as fórmulas de covariância, correlação e regressão.
- O módulo básico é reutilizado para o cálculo das médias. Além de calcular as médias, be_media também valida o ponteiro e valores.
- A covariância calculada é a populacional (divide por n).
- A regressão reaproveita calcular_corr, pois r2 é correlação ao quadrado. Por conta disso, be_regressao_linear não precisa tratar o caso de divisão por zero diretamente, pois isso é feito em calcular_corr.
- constante faz uma comparação exata com valores. A comparação com as médias pode dar errado devido à aproximações.

17.4.4 Tratamento de erros e casos de borda

- O ponteiro de saída é verificado primeiro. Depois, verifica-se os ponteiros, os valores e a divisão por zero, nessa ordem.

- Divisão por zero pode ocorrer quando sxx * syy é zero. Isso ocorre quando uma das duas variáveis é constante.

- Não há erro de memória, já que esse módulo não aloca memória.

- Casos de borda tratados:
  - n == 1: Covariância vale 0. Correlação e regressão devolvem erro de divisão por zero.
  - n == 2: Correlação é 1 ou -1.
  - Alguma variável é constante (n == 1 cai dentro desse caso): Covariância vale 0. Correlação e regressão devolvem erro de divisão por zero.
  - Sem relação linear: Covariância e correlação nulas.
  - Saídas intactas em caso de erro.

17.4.5 Dificuldades encontradas

- Verificar o status das chamadas internas.
- Séries constantes e arredondamentos. Testar den == 0.0 não era o suficiente por conta de arredondamentos.
- Regressão com dados_y constante. r2 fica indefinido.

17.4.6 Testes

- Os testes abordam:
  - Casos com entrada normal;
  - Casos de borda;
  - Casos com ponteiro nulo;
  - Casos com valor inválido;

17.5 Módulo de visualização

17.5.1 Visão geral do Módulo

O objetivo desse módulo é oferecer suporte a geração de gráficos tanto rapidamente no terminal através de ASCII, quanto de maneira mais polida por arquivos vetoriais (SVG/HTML), para visualização no navegador.

17.5.2 O que foi desenvolvido

- Structs:
  - Canvas, para agrupar o ponteiro da matriz 2D, as dimensões e offset dela, reduzindo a complexidade na hora de manipular os desenhos em ascii.
  - Quartis, para agrupar os 3 quartis necessários para o boxplot e também reduzir a complexidade na hora de manipular esses dados dentro das funções.
- Funções de desenho em ASCII:
  - be_dispersao_ascii: Desenha o gráfico de dispersão, mapeando pares (Xi, Yi). Desenha esses pontos com o caracter ’*’ na matriz 2d, ajusta as medidas de acordo com a escala da matriz e gera uma legenda com o tamanho da amostra, e os range de valores que cada variável assume.
  - be_histograma_ascii: Desenha um histograma, baseado nos dados e número de intervalos inseridos. Preenche as barras com ‘#’ na matriz 2d, ajusta as medidas de acordo com a escala da matriz e gera uma legenda com informações de cada intervalo.
  - be_boxplot_ascii: Desenha um boxplot, baseado nos dados e nos quartis inseridos. Preenche os “bigodes”, a caixa principal, a mediana e desenha os outliers com o caracter ’*’. Ajusta todas medidas de acordo com a escala da matriz e gera uma legenda com os valores de cada ponto crítico do boxplot.
- Funções de desenho em SVG/HTML:
  - be_dispersao_svg: Gera um arquivo html de nome escolhido contendo o gráfico de dispersão, mapeando pares (Xi, Yi). Desenha esses pontos com a tag `<circle>` e gera uma legenda mais completa que sua versão ASCII. Os eixos são desenhados através da tag `<line>`, usando valores da função para localizá-los adequadamente. Isso também vale para as demais tags.

    - be_histograma_svg: Gera um arquivo html de nome escolhido contendo o histograma, baseado nos dados e número de intervalos inseridos. Usa a tag `<rect>` para gerar as barras, usando valores da função para localizá-los adequadamente. Isso também vale para as demais tags. Também conta com rótulos adequados como densidade de frequência no eixo y, e intervalos no eixo x.
    - be_boxplot_svg: Gera um arquivo de html de nome escolhido contendo o boxplot, baseado nos dados e nos quartis inseridos. Utiliza a tag `<rect>`, `<line>`, `<circle>`, para a caixa principal, eixos, bigodes, linhas e outliers, que por sua vez usam valores da função para localizá-los adequadamente. Isso também vale para as demais tags. Também conta com os a localização no eixo x de cada ponto crítico escrito nesses locais no gráfico.

- Funcões auxiliares (privadas):

    - esvaziaCanvas: função que limpa a matriz 2D de chars onde o gráfico fica totalmente.
    - desenhaEixos: função que desenha os eixos na matriz 2D de chars.
    - imprimeCanvas: função que imprime no terminal a matriz 2D do gráfico.
    - achaMin: função que acha o valor mínimo num array dado.
    - achaMax: função que acha o valor máximo num array dado.
    - normaliza: função que pega um valor, o maior valor, o menor valor, calcula qual porcentagem relativa do valor dada a amplitude de valores disponíveis, e multiplica pela escala inserida, assim normalizando um valor à matriz por exemplo.

17.5.3 Decisões de arquitetura e design

- O módulo funciona de modo completamente independente, necessitando apenas dos dados de entradas das funções, e devolvendo o canvas ou svg pronto.
- Os eixos x e y são dinâmicos, isto é, se movem de acordo com a origem quando necessário, para uma representação menos confusa dos valores.
- Integração com a convenção global dessa biblioteca de tratamento de erros, o enum be_status, fazendo cada função retornar um status (como BE_OK), facilitando a localização de um erro em sua ocorrência, ou de um sucesso.

17.5.4 Tratamento de erros e casos de borda

- Todos ponteiros que entram nas funções desse módulo possuem checagem para saber se eles são referências reais ou apenas nulas, evitando que o C retorne um erro mais crítico de memória.
- Sempre que existe a possibilidade de ocorrer uma divisão por zero, esses casos são ou evitados no começo da função, ou contornados com checagem individual de valores e atribuição de um valor válido.
- Para uso de doubles, onde preciso existe uma checagem de precisão, para evitar por exemplo que um vetor seja acessado através de um índice não inteiro.
- Sempre que criado um arquivo logo em seguida é verificado se a criação realmente ocorreu, antes de qualquer operação dentro dele.

17.5.5 Dificuldades encontradas

- Problemas com o uso de calloc e com a liberação da memória no momento errado.
- Muitos problemas com a não inicialização de variáveis, que por ficar com lixo na memória acabam causando valores indesejados.
- Inviabilidade de dentro da matriz escrever os valores dos pontos ou colocar rótulos nos eixos, a alternativa foi simplesmente legendar esses gráficos ASCII.
- Se adaptar da visão do desenho na mão para o desenho em coordenadas x e y tanto no ASCII quanto no SVG, e tornar isso de certa forma abstrato para qualquer valor ficar no lugar certo.
- Fazer os eixos serem relativos a posição da origem quando necessário, pois exige um recálculo geral.
- Confusão que a altura são as linhas (o eixo y é escrito a partir de um x fixo) e a largura as colunas (o eixo x é escrito a partir de um y fixo).
- Acertar os caracteres para uma visualização limpa no ASCII e no SVG acertar as cores e formas.

17.5.6 Testes

- Existem testes unitários em tests/test_visualizacao.c, utilizando assert, de cada uma das funções, testando valores de borda, nulos, se para valores válidos os gráficos ASCII são desenhados corretamente e se para o SVG os arquivos estão sendo criados corretamente e não estão vazios.